

Phase 1 — Measurement + Technical Foundations

Derived from: docs/superpowers/specs/2026-09-30-seo-aicorebots-design.md (§6) Scope: Option B — no blog. This plan covers Phase 1 only. Estimated effort: half a day.

All file paths and line numbers in this plan were read from the repository on 2026-09-30 at commit 913c4bd. If a line reference does not match, the file changed — re-read before editing.

How this plan is organized

Tasks are split into two groups, and the split is the point:

Group A — Structural. Mechanical, deterministic, verifiable by command. No copy is invented, no wording is chosen, no design decision is needed. A subagent can execute these end to end and prove the result.

Group B — Editorial. Requires a human decision about wording or appearance. These are marked BLOCKED-ON-OWNER and must not be executed by a subagent.

Group A is the unblocking work. Group B does not block Group A, and nothing in Group A depends on Group B.

Group A — Structural tasks

A1. robots.txt — explicit AI-crawler permissions

File: robots.txt (4 lines)

Current state, verbatim:

```
User-agent: *  
Allow: /
```

Sitemap: <https://aicorebots.com/sitemap.xml>

There is no Disallow anywhere, so the site is already crawlable by default. This task adds named, explicit allow blocks for the crawlers that matter for AI-citability, and does not change the `User-agent: *` group.

Add these four groups before the `Sitemap:` line, each with `Allow: /`:

- GPTBot (OpenAI training + ChatGPT search)
- OAI-SearchBot (OpenAI search index — distinct from GPTBot)
- ClaudeBot (Anthropic crawler)
- PerplexityBot (Perplexity index)

Also add a short `User-agent: Claude-User` group with `Allow: /`. This is the user-triggered fetch agent, distinct from ClaudeBot, and it is the one that actually retrieves pages when a user pastes a link.

Verification:

`Get-Content robots.txt`

Assert: 5 new `User-agent:` groups present, each with `Allow: /`, plus the original `User-agent: *` group unchanged, plus the `Sitemap:` line last. No Disallow added.

A2. llms.txt + llms-full.txt — new root files

Files: do not exist. Create both.

llms.txt is a plain-text map of the site for language models, following the llmstxt.org convention: an H1 with the site name, a blockquote summary, then link sections.

Target content:

- H1: # aicorebots.com
- Summary line: what the company does, in one sentence, no marketing adjectives
- Section ## Productos with the five product routes and one factual line each: /productos/aicoremed/, /productos/captacion-360/, /productos/portal-paciente/, /productos/aicorelink/, /productos/iapo/
- Section ## Informacion with /faq/, /privacidad/, /terminos/, /aviso-legal/

Derive every description from text that already exists on the site. Do not write new claims. If a one-line description cannot be derived from existing page copy, omit the line rather than inventing one.

llms-full.txt is the full-text version: the readable body content of each route in plain text, headings intact, no navigation, no footer boilerplate.

Why this task is coupled to A3: both files are new root files, and the Dockerfile lists root files explicitly. Writing them without fixing the Dockerfile means they never reach the deployed image.

Verification:

`Get-ChildItem llms.txt, llms-full.txt | Select-Object Name, Length`

Assert: both exist, both non-empty, llms.txt starts with # aicorebots.com.

A3. Dockerfile — ship the new root files

File: Dockerfile (11 lines)

Line 3, verbatim:

```
COPY index.html 404.html robots.txt sitemap.xml /usr/share/nginx/html/
```

Line 4, verbatim:

```
COPY productos/ /usr/share/nginx/html/productos/
```

Root files are listed explicitly. llms.txt and llms-full.txt are not in that list, so they would be silently absent from the built image — a 404 to every crawler that requests them, with no error anywhere.

Change: add llms.txt llms-full.txt to line 3.

Do not change line 4. It is already recursive, so the Phase 2 product subdirectories will be picked up with no edit. Do not “helpfully” list them.

Verification:

`Select-String -Path Dockerfile -Pattern "llms"`

Assert: line 3 now contains both filenames, in the same COPY instruction. Assert line 4 is byte-identical to the current value.

A4. sitemap.xml — real lastmod, drop ignored fields

File: sitemap.xml (39 lines)

Current state: 6 <url> entries, **every one** carrying <lastmod>2026-09-30</lastmod>, and every one carrying <changefreq> and <priority>.

Two problems. The dates are fiction — a flat stamp for every URL tells a crawler nothing and is the kind of inconsistency that erodes trust in the signal. And Google has ignored changefreq and priority since 2015; they are dead weight in the file.

Verified real dates from `git log -1 --format=%ad --date=format:%Y-%m-%d`:

URL	Real lastmod
https://aicorebots.com/	2026-09-30
https://aicorebots.com/productos/	2026-09-29
https://aicorebots.com/faq/	2026-09-29
https://aicorebots.com/privacidad/	2026-09-29
https://aicorebots.com/terminos/	2026-09-29
https://aicorebots.com/aviso-legal/	2026-09-29

Changes: set each `<lastmod>` to its real git date per the table. Remove every `<changefreq>` and `<priority>` element.

Constraint: after this commit, `index.html` changes in Group A (A6, A7), so its real lastmod becomes the date of that commit. Re-read the git date for `index.html` at the end of Group A and update the `/` entry to match. Do not predict it — read it.

Verification:

`Select-String -Path sitemap.xml -Pattern "changefreq|priority"`

Assert: zero matches. Assert 6 `<lastmod>` elements remain, one per `<url>`, each matching the git date for its file.

A5. 404.html — remove the unreachable canonical

File: 404.html (45 lines)

Line 9 already contains:

```
<meta name="robots" content="noindex,follow">
```

That part is correct and needs no change. (Note: no space after the comma, differing from spec §6.5. Both forms are valid; leave it alone rather than creating a diff for cosmetics.)

Line 11 is the actual bug:

```
<link rel="canonical" href="https://aicorebots.com/404.html">
```

This canonical is unreachable. `nginx.conf` lines 26-29 contain `location = /404.html { internal; ... }`, so the file is served only as an error body and can never be fetched at its own URL. A canonical pointing at a URL that returns 404 is a contradiction; the correct move is to remove the tag entirely. A 404 page has no canonical to declare.

Change: delete line 11.

Verification:

`Select-String -Path 404.html -Pattern "canonical"`

Assert: zero matches. Assert the `noindex,follow` meta on line 9 is intact.

A6. index.html <head> — missing social and robots tags

File: index.html, head region lines 1-36

Verified present: title (6), description (7), `og:title` (10), `og:description` (11), `og:url` (12), `twitter:card` (13), canonical (14), `og:image` (16) with declared 1200x630 (17-18), `twitter:image` (19), preconnects (20-21), Google Fonts stylesheet (22), JSON-LD (24-34).

Verified absent: `og:site_name`, `og:locale`, `twitter:site`, `<meta name="robots">`.

Add four tags to the head:

- `<meta name="robots" content="index,follow,max-image-preview:large,max-snippet:-1,max-video-preview:-1">` — the `max-image-preview:large` is the one with real effect: without it Google may serve a thumbnail-sized image in image results.
- `<meta property="og:site_name" content="aicorebots">` — use the brand string as it already appears in the site, lowercase, no `.com`.
- `<meta property="og:locale" content="es_CL">` — Chile, from the Viña del Mar address already in the JSON-LD.
- `<meta name="twitter:site" content="@aicorebots">` — **BLOCKED**, see **B3**. Place a placeholder-free version only once Leonardo confirms the handle exists. If unconfirmed, omit this tag in this task and handle it in **B3**.

Verification:

Select-String -Path `index.html` -Pattern `"og:site_name|og:locale|max-image-preview"`

Assert: three matches, one each.

A7. `index.html` JSON-LD — complete the Organization node

File: `index.html`, @graph lines 24-34

The graph currently has 7 nodes: Organization (`#org`, line 26), Website (`#website`, line 27), and five Service nodes (`#svc-aicoremed` 28, `#svc-cap360` 29, `#svc-portal` 30, `#svc-link` 31, `#svc-iapo` 32).

The `#org` node is thin. Verified gaps:

- `logo` is a bare URL **string**, not an `ImageObject`. Schema.org accepts either, but the `ImageObject` form carries width/height and is what Google's rich-result validator prefers.
- `address` has only `addressLocality` and `addressCountry`. No `streetAddress`, no `postalCode`.
- No `telephone`, no `email`, no `contactPoint`, no `foundingDate`, no `areaServed`.

Verified contact data, all confirmed against the site and by Leonardo:

Field	Value	Source
<code>streetAddress</code>	1 Oriente 714	present on site
<code>addressLocality</code>	Viña del Mar	present on site
<code>email</code>	<code>contacto@aicorebots.com</code>	present on site
<code>telephone</code>	+56 9 7568 0702	Leonardo confirmed as the public contact number

The old spec number `+56 9 51805423` does not appear anywhere on the current site and must never be used. It was not verified.

Logo dimensions: `assets/logo.svg` declares `width="1467" height="389"`. Use those exact values in the `ImageObject`.

Add to `#org`:

- `logo` → convert the bare string into an `ImageObject` with `url`, `width`: 1467, `height`: 389, and `caption`: "aicorebots"
- `telephone`: +56 9 7568 0702
- `email`: `contacto@aicorebots.com`
- `foundingDate`: **BLOCKED**, see **B4**. Do not invent a founding year.
- `address.streetAddress`: 1 Oriente 714

- `areaServed`: Viña del Mar, Valparaiso Region, Chile — derive the region from the city; the site already says Viña del Mar
- `contactPoint` with `@type`: `ContactPoint`, `contactType`: `"customer service"`, `telephone`: `" +56 9 7568 0702"`, `email`: `"contacto@aicorebots.com"`, `areaServed`: `"CL"`

Do not add: `AggregateRating`, `Review`, `priceRange`, or a Chilean RUT / razón social. There are no real reviews; fabricated rating markup is a manual action risk, not a ranking win. Prices are not public.

Verification: every `@id` reference inside the graph must resolve to a node that exists. No dangling refs.

A8. `index.html` — add `BreadcrumbList` and `WebPage` nodes

File: `index.html`, `@graph`

The graph has no `BreadcrumbList` and no `WebPage` node. Only the home page needs these in Phase 1 — the product routes arrive in Phase 2 with their own.

Add `WebPage` (`#webpage`): `@type`: `WebPage`, `@id` matching the page canonical URL, `url`, `name` matching the existing `<title>`, `isPartOf` pointing at the existing `#website` node, `about` pointing at `#org`, `inLanguage`: `"es-CL"`, and `breadcrumb` pointing at the new breadcrumb node.

Add `BreadcrumbList` (`#breadcrumb`): the home page has exactly one item.

```
{
  "@type": "BreadcrumbList",
  "@id": "https://aicorebots.com/#breadcrumb",
  "itemListElement": [
    {
      "@type": "ListItem",
      "position": 1,
      "name": "Inicio",
      "item": "https://aicorebots.com/"
    }
  ]
}
```

A one-item breadcrumb is valid and is what a single-level hierarchy produces. Do not invent a second level.

Verification: all new `@id` refs resolve; `#webpage.isPartOf` → `#website` exists; `#webpage.breadcrumb` → `#breadcrumb` exists.

Group B — Editorial tasks, owner decision required

None of these block Group A. Each needs Leonardo's input before execution.

B1. speakable on question-answering H1s — BLOCKED

Which headings answer a question, and therefore deserve `speakable`, is a content judgment. The property is only valid on a `WebPage` node and must point at CSS selectors whose content matches the visible text.

Related risk discovered during verification, which must be resolved together with this decision:

`assets/site.js` lines 14-22 process every `[data-wave]` element by reading `h.textContent`, setting `aria-label` to it, then **wiping `h.textContent` and rebuilding the heading as per-character `<span>s`** — words in `.wd`, letters in `.ch` with a `--i` stagger index.

Verified detail: every rebuilt span carries `aria-hidden="true"`, so after the script runs the visible heading text is entirely hidden from assistive technology and the only accessible copy is the `aria-label`. This does not affect crawlers, because the raw server-rendered HTML still carries the real H1 text — the fact lives in the HTML and

the script only re-renders it, which is what spec §11 requires. It is recorded here so nobody later “simplifies” this by stripping `aria-label` on the assumption that the visible text survives.

Consequences for any H1 change on `index.html` or `productos/index.html`:

- A long search-term H1 renders at `h1{font-size:clamp(3rem,7.6vw,7.4rem)}`, which is sized for a four-word headline. A search-term H1 such as “Agentes de IA para clínicas y negocios que atienden por WhatsApp” will overflow at that size.
- `.wd` is `display:inline-block;white-space:nowrap`, so no word breaks internally; only the wrap points move.
- `text-wrap:balance` is already active on `h1`, which helps, but it cannot fix a single word wider than the viewport.

`faq/index.html` H1 has **no** `data-wave` attribute, so it is untouched by that script — it can change freely.

Leonardo decides: whether the search-term H1s replace the current brand-voice ones, and if so whether `site.css` needs a font-size reduction for those specific headings, or whether a shorter search-term phrasing is used instead. This is the H1 sacrifice the spec flags as intentional and reversible per page.

Current H1s, for reference:

- `index.html:61` — `<h1 data-wave>Tu negocio, con pulso propio.</h1>`
- `productos/index.html:57` — `<h1 data-wave>Cinco herramientas, un mismo pulso.</h1>`
- `faq/index.html:63` — `<h1>Lo que más nos preguntan</h1>`

Blocker on B1: which H1s change, and with what wording. A subagent must not invent search terms.

B2. `BreadcrumbList` and `WebPage` on the remaining routes — **BLOCKED**

Phase 1 adds these to the home page only. The other five routes (index plus five legal/FAQ pages) get the same treatment in Phase 2 or Phase 3, once the route structure settles. Confirm the split before extending.

B3. `twitter:site` handle — **BLOCKED**

A6 adds `og:site_name`, `og:locale`, and the `robots` meta, but omits `twitter:site` because the handle must be a real, existing account. A wrong handle is worse than an absent one. Leonardo confirms the handle, or it stays omitted.

B4. `foundingDate` — **BLOCKED**

A7 adds the `Organization` fields that are verified. `foundingDate` requires the actual founding year, which is not on the site and not inferable. A guessed year in schema is a factual error that outlives this project. Omit it until confirmed.

Deliberately not in Phase 1

- **No blog.** `/blog/` 301s to `iapo.cl`. Owner decision, not revisited here.
 - **No analytics, no cookie banner.** Server-side Search Console only.
 - **No `AggregateRating` / `Review`.** No real reviews exist.
 - **No product route pages.** Those are Phase 2.
 - **No FAQ expansion.** 8 questions today, 25 in Phase 3.
 - **No framework migration.** Static server-rendered HTML stays static; no client-side injection of indexable content.
-

Manual steps, not automatable

Search Console

Search Console is the measurement baseline for everything that follows. Without it, later phases cannot be attributed to anything.

It is **not blocking** for Group A. Google retains roughly 16 months of historical data, so registering after deploy still shows the “before” state as long as it is not too many weeks later. What is lost is the granularity of when each change took effect.

Leonardo verifies the `aicorebots.com` property and confirms ownership. This cannot be scripted — it needs a browser and a Google account.

Note the priority: because the site is server-rendered and sends no cookies, Search Console is a clean fit. No consent banner is required, which keeps the no-banner decision intact.

Verification protocol for this plan

Per spec §11, the whole phase must satisfy all of these before it is considered done:

1. Every JSON-LD block parses as valid JSON.
2. Every `@id` reference inside every graph resolves to an existing node.
3. No indexable claim exists only in JavaScript. The site is server-rendered; `site.js` may animate but must not be the source of a fact.
4. `docker build` succeeds.
5. The built image serves `llms.txt` and `llms-full.txt` at the root — verified from inside the image, not from the source tree. The source tree having the files proves nothing about the image.
6. Sitemap `lastmod` values match real git dates, read after the final commit.
7. No `canonical` remains in `404.html`.
8. No `changefreq` or `priority` remains in `sitemap.xml`.

Check 5 is the one most likely to be skipped and the one that actually matters. A file in the source tree that never entered the image is invisible to every crawler, and nothing in the build fails when that happens.